Parallel Graph Analytics of the Hong Kong Film Industry

Co-star Networks, Genre Associations and Talent Flow from 180 Million IMDb Records

CS5488 Data-Intensive Computing — Group Project Proposal

November 2026

Student ID	Student Name	Student Email
[Student ID 1]	[Student Name 1]	[Email 1]
[Student ID 2]	[Student Name 2]	[Email 2]
[Student ID 3]	[Student Name 3]	[Email 3]
[Student ID 4]	[Student Name 4]	[Email 4]

1. Abstract

The Hong Kong film industry is one of the densest collaborative networks in cinema history: a small pool of actors, directors, action choreographers and cinematographers reused across thousands of productions, organised first around the studio system and later around independent production companies. This project reconstructs that collaboration network from the complete IMDb public dataset — approximately 180 million rows spanning title metadata, regional release titles, principal cast and crew credits, and personal records — and analyses its structure with parallel algorithms. We address four questions: who the structural hubs of Hong Kong cinema are; whether unsupervised community detection recovers the historical studio and production-company blocs; which genre–crew combinations co-occur beyond chance; and how communities recombine across five decades. Methodologically, the contribution is a three-tier parallel design that matches each workload to an appropriate parallel strategy: a chunk-level map-reduce scan for the 180-million-row ingestion and join stage, blocked sparse matrix–vector multiplication for iterative PageRank and label propagation on the co-star graph, and parallel candidate generation plus lock-free counting for association rule mining. We benchmark every tier against a single-threaded baseline, reporting strong scaling from 1 to 16 workers, parallel efficiency, peak resident memory, and the effect of block and chunk size on throughput, and we locate the crossover point at which inter-worker communication outweighs the gain from additional cores. The result is both a reproducible parallel pipeline and an empirical account of how Hong Kong's film community structure changed between the 1970s and the 2020s.

Keywords: parallel graph analytics, PageRank, community detection, association rule mining, map-reduce, memory hierarchy, Hong Kong cinema

2. Introduction

2.1 Motivation

Hong Kong produced roughly 8,000 feature films in the post-war period, and for most of that time the industry operated on a small, intensely reused labour pool. Shaw Brothers and Golden Harvest each maintained contracted stables of actors and technicians; the New Wave and later Milkyway Image worked with recurring ensembles. The consequence is an unusually dense, unusually traceable co-appearance network: the same faces recur across genres and decades, and studios leave identifiable fingerprints in who works with whom.

Most published work on film collaboration networks studies Hollywood or global cinema in aggregate, where the network is large but shallow — individual collaborations are sparse and communities are diffuse. Hong Kong is the opposite: a compact network with strong, historically documented substructure. That makes it an ideal test bed for asking whether unsupervised graph algorithms recover structure that film historians already assert, and whether the parallel implementation needed to compute it at scale is worth its complexity.

From a data-intensive computing standpoint the problem is attractive because the compute is genuinely heavy while the input is clean. No scraping, no schema reconciliation, no API quotas: a handful of compressed tab-separated files, refreshed daily, that together require a full scan of roughly 180 million rows to answer a single query.

2.2 Objective

The project has four objectives, each pairing a substantive research question with an engineering one.

O1 — Reconstruct the network at scale. Ingest the full IMDb non-commercial dataset, filter to Hong Kong–region titles, and build a weighted, time-stamped co-star graph. Engineering goal: a chunk-parallel ingestion and join pipeline that processes the full 180 million rows without exhausting memory on a commodity laptop.

O2 — Characterise network structure. Compute degree, PageRank and sampled betweenness centrality on the co-star graph to identify structural hubs, distinguishing actors who are merely prolific from actors who bridge otherwise disconnected communities. Engineering goal: a blocked sparse matrix–vector implementation whose block size is chosen from measured cache behaviour rather than guesswork.

O3 — Recover latent communities and their evolution. Apply label propagation and Louvain modularity optimisation, then track community membership by decade to test whether detected communities align with documented studio affiliations and how they fracture after the industry's early-1990s contraction. Engineering goal: measure how community detection parallelises in practice, where iterative convergence limits achievable speedup.

O4 — Mine genre and crew associations. Treat each film as a market basket of genre labels and crew roles, and mine association rules at multiple support thresholds, comparing the parallel cost of Apriori-style candidate generation against FP-Growth. Engineering goal: quantify how load imbalance in candidate generation degrades parallel efficiency, and demonstrate work-stealing or dynamic scheduling as the remedy.

2.3 Data Sources

All data comes from the IMDb non-commercial datasets, published at `https://developer.imdb.com/non-commercial-datasets/` as gzip-compressed tab-separated files. The release is refreshed daily; row counts below are approximate and must be recorded at download time for reproducibility.

File (`.tsv.gz`)	Approx. rows	Fields used	Role in this project
`title.basics`	11.8 M	`tconst`, `titleType`, `startYear`, `genres`, `runtimeMinutes`	Film identity, decade bucketing, genre basket
`title.akas`	51 M	`titleId`, `region`, `language`, `isOriginalTitle`	Hong Kong region filter; alternative-title joins
`title.principals`	90 M	`tconst`, `nconst`, `category`, `job`, `characters`, `ordering`	Cast and crew edges; billing order
`name.basics`	14.5 M	`nconst`, `primaryName`, `birthYear`, `primaryProfession`	Node attributes; career-span features
`title.ratings`	1.5 M	`tconst`, `averageRating`, `numVotes`	Quality weighting; minimum-vote filters

Two supporting sources are used only for validation, not for analysis: the Hong Kong Film Archive catalogue (`https://www.filmarchive.gov.hk/`) to hand-check a sample of 50 detected communities against documented studio affiliations, and the Hong Kong Movie Database (`https://hkmdb.com/`) to verify titles whose IMDb region tagging is ambiguous.

A note on the region filter. `title.akas` carries one row per regional release, so a Hong Kong production typically appears with `region = HK` alongside `CN`, `TW` and others. We define the working set as titles having at least one `HK` alias and `titleType` in {`movie`, `tvMovie`, `short`}. This is a heuristic, not ground truth: co-productions and pre-1970 titles are unevenly tagged. Section 5.3 treats this as a threat to validity and reports a sensitivity check using an alternative definition (original language `cn` or `zh` plus a Hong Kong production company in `title.crew`).

3. Methodology

3.1 Pipeline architecture

The pipeline has five stages, each a separate executable so that stages can be benchmarked and cached independently. Intermediate results are written as Parquet so that a failed experiment does not require re-scanning 180 million rows.

Stage	Input	Operation	Output
S0 Ingest	5 `.tsv.gz` files	Streaming decompress, schema validation, column pruning	Parquet shards, 64 MB each
S1 Filter and join	Parquet shards	Region filter, three-way join on `tconst`, decade bucketing	Hong Kong title table
S2 Graph build	Hong Kong title table + principals	Per-title cast combination, edge weighting, self-loop removal	Edge list + node table
S3 Graph analytics	Edge list	PageRank, label propagation, Louvain, sampled betweenness	Node scores, community labels
S4 Rule mining	Genre and crew baskets	Frequent itemset generation, rule evaluation	Rule table with support, confidence, lift
S5 Analysis and figures	All outputs	Statistical tests, aggregation, plotting	Report figures

Stages S0 to S2 are data-parallel with no cross-chunk dependency, which is what makes near-linear speedup achievable. Stages S3 and S4 contain iterative or irregular dependencies, and are where parallel efficiency is genuinely contested — this contrast is the analytical backbone of Section 4.

3.2 Parallel strategy

The design deliberately uses three different strategies rather than one, because the three workloads stress different parts of the memory hierarchy.

Tier 1 — Chunk-level map-reduce for ingestion and join (S0–S2). Each worker takes a contiguous byte range of the decompressed stream, parses it independently, applies the region filter, and emits a partial result. Partial results are combined by a single reduction step. This is the textbook map-reduce pattern, and it was chosen for a specific reason: the stage is I/O- and parse-bound, and Python's global interpreter lock prevents threads from overlapping the parse work. Process-based workers each hold their own interpreter, so the ten to fourteen cores on a typical laptop are actually used.

Chunk size is treated as a tuning parameter rather than a constant. Small chunks (1–4 MB) improve load balance and keep the working set inside L2/L3 cache, but increase per-task overhead and the number of partial results the reducer must merge; large chunks (64–256 MB) reduce overhead but leave cores idle

at the tail of the run. Section 4.1 reports a chunk-size sweep and justifies the selected value from measured throughput, which is precisely the memory-hierarchy argument the marking scheme asks for.

Tier 2 — Blocked sparse matrix–vector multiplication for PageRank and label propagation (S3). The co-star graph is represented as a sparse adjacency matrix in compressed sparse row form, with roughly one million nodes and ten million edges after billing-order truncation. PageRank is a sequence of sparse matrix–vector products, so the question is not how to parallelise it — rows partition cleanly — but how to keep the vector in cache and how to limit synchronisation.

We implement an owner-computes row-block decomposition: the adjacency matrix is split into row blocks, each worker multiplies its block against the current vector and writes a private partial vector, and blocks are synchronised only at iteration boundaries. Block size is chosen so that one row block plus the destination vector fits within a target cache level, which reduces cache misses on the gather of the source vector. We report iteration time against block size and against thread count, and we measure the fraction of runtime attributable to the barrier between iterations to quantify Amdahl's limit for this stage.

Tier 3 — Parallel candidate generation with lock-free counting for association rules (S4). Apriori-style mining has two phases with opposite parallel characteristics. Candidate generation is irregular — the cost per candidate depends on how many transactions contain it — so a static partition produces severe load imbalance and dynamic scheduling is required. Counting is regular and almost perfectly parallel, because each worker can accumulate local counters over its transaction shard and merge once at the end without any locking during the scan. Measuring the two phases separately, and showing that the imbalance lives entirely in the first, is the empirical point of Section 4.3.

For all tiers we report wall-clock time on 1, 2, 4, 8 and 16 workers, computing speedup $S(p) = T(1)/T(p)$ and parallel efficiency $E(p) = S(p)/p$. Strong scaling doubles the worker count at fixed data size; weak scaling doubles both data size and worker count. Peak resident memory is sampled per worker so that the memory cost of parallelism is reported alongside the time benefit — a trade-off that students frequently omit and examiners frequently ask about.

3.3 Analysis methods

Centrality. Degree counts collaborations but conflates prolificacy with importance. PageRank on the undirected, vote-weighted graph corrects for this by propagating importance through the collaboration structure; a veteran supporting actor who worked with many leads accumulates rank without appearing in the top degree list. Exact betweenness is $O(V \cdot E)$ and intractable at this scale, so we use Brandes with $k = 512$ pivot sources and report sampling error against a $k = 2{,}048$ run.

Community detection. Label propagation is used as the fast, fully parallel baseline; Louvain modularity optimisation provides the quality reference. Because Louvain is stochastic, each configuration is run 20 times and modularity is reported as mean ± standard deviation. Agreement between runs is measured by normalised mutual information, which also serves as a stability check on the detected communities.

Temporal analysis. The graph is sliced into five decades (pre-1980, 1980s, 1990s, 2000s, 2010s onward) using `startYear`. Rather than re-running detection per slice in isolation, we fix the community labels from the full graph and measure membership flux per decade — the share of a community's members active in each decade. This shows community birth, decline and recombination without the instability of comparing independently detected partitions across slices.

Association rules. Each film is a basket containing one item per genre plus one item per principal crew category (`director`, `writer`, `cinematographer`, `composer`, `production designer`, `actor`).

Frequent itemsets are mined with minimum support swept over {0.5%, 1%, 2%, 5%} of films; rules are retained at confidence ≥ 0.6 and lift > 1.2, and evaluated with a chi-square test on the corresponding contingency table to separate genuine association from small-sample artefacts in sparse early-era data.

Clustering. Career trajectories are clustered with k-means on features derived from each person's credits: career length, credits per active year, genre entropy, mean billing order, and decade of peak activity. k is selected by silhouette score over k = 2…12. This provides a complementary, non-graph view of the same population and lets us check whether graph communities correspond to career types.

4. Results and Analysis (Planned)

4.1 Experiment design

Four core experiments, plus a scaling study that cross-cuts all of them. Each experiment is run five times; we report median wall time and interquartile range.

ID	Research question	Baseline (single core)	Parallel implementation	Metrics and expected outcome
E1	How fast can 180 M rows be filtered and joined?	`pandas.read_csv` with `chunksize`, one process	a process pool over byte ranges, Parquet write per worker	Wall time, peak RSS, chunk sweep 1–256 MB. Expect 5.0–6.5× at 8 workers; throughput peaks near L3 capacity, then flattens
E2	Does graph construction scale linearly?	Single-pass edge accumulation in a dict	Partitioned build keyed by `tconst`, per-worker edge shards, single merge	Wall time, edge count, merge-cost share. Expect 6–7.5× at 8 workers; the merge becomes a visible serial tail
E3	Where does PageRank stop scaling?	networkx PageRank	Owner-computes row-block sparse matvec, barrier per iteration	Time per iteration, iterations to converge, block sweep, barrier share. Expect 4–6× at 8 workers; efficiency below 0.5 beyond 8 as the barrier dominates
E4	Is rule mining limited by counting or by candidate generation?	mlxtend Apriori	Parallel candidate generation with dynamic scheduling, plus lock-free shard counting	Time per phase, rule count, support sweep, load-imbalance ratio. Expect counting above 7× at 8 workers, candidate generation saturating near 3×

S-series scaling study. For each of E1–E4 we report strong scaling at $p \in \{1, 2, 4, 8, 16\}$ and weak scaling by doubling the working set. Two derived quantities are reported for every experiment: parallel efficiency E(p), and the serial fraction implied by a fitted Amdahl model. The comparison across E1–E4 — four workloads of deliberately different parallel character on identical hardware — is the core technical contribution, because it converts a set of isolated speedup numbers into a claim about which workload shapes are worth parallelising on commodity hardware.

4.2 Figure inventory

Fig.	Content	Source	Section
1	Pipeline architecture, five stages with parallel tiers annotated	Diagram	3.1
2	Row counts and join fan-out per dataset, as a flow diagram	S0 logs	3.1
3	Films per decade, Hong Kong working set	S1 output	4.1
4	Degree distribution on log–log axes with fitted power-law tail	S3 output	4.1
5	PageRank versus degree scatter, hubs labelled	S3 output	4.2
6	Co-star network, top 500 nodes by PageRank, coloured by community	S3 output	4.2
7	Community composition table, detected versus documented studio blocs	S3 + manual check	4.2
8	Community membership flux across five decades (stacked area)	S3 temporal	4.2
9	Genre–crew association rules, support versus lift, top 30 labelled	S4 output	4.3
10	Strong scaling curves for E1–E4 on one axis	E-series	4.4
11	Parallel efficiency versus worker count, with Amdahl fit	E-series	4.4
12	Chunk-size and block-size sweeps against throughput	E1, E3	4.4
13	Peak memory per worker versus worker count	E-series	4.4
14	Career-type clusters projected on two principal components	Clustering	4.5

Ten to fourteen figures is the right density for a report of this length: enough to carry the analysis, not so many that each is under-explained. Figures 10 to 13 are the ones that earn the parallel-computing marks; Figures 4 to 9 are the ones that earn the technical-depth marks.

5. Discussion (Planned)

5.1 Trade-offs to be quantified

Three trade-offs will be measured rather than asserted. First, **exactness versus sampling** in betweenness centrality: the k-pivot approximation must be shown to preserve the ranking of the top 50 nodes, or the hub analysis is not trustworthy. Second, **quality versus speed** in community detection:

label propagation is far cheaper than Louvain, and the modularity gap quantifies what that speed costs. Third, **time versus memory** in parallel execution: more workers shorten wall time but multiply peak resident memory, and on a laptop the binding constraint is frequently memory rather than cores — a result worth reporting explicitly because it contradicts the naive assumption that more workers is always better.

5.2 Expected bottlenecks

We anticipate that stage S1 will be I/O-bound until the data is in Parquet, after which it becomes CPU-bound; that PageRank will be limited by the inter-iteration barrier once per-iteration work falls below roughly 100 ms; and that candidate generation in E4 will be limited by load imbalance and by the size of the second-pass candidate set. Each of these is a testable prediction, and the report will state whether the measurement confirmed or refuted it.

5.3 Threats to validity

The region filter is a heuristic built on alias metadata that is unevenly maintained before 1970 and for co-productions; a sensitivity analysis with an alternative definition is reported alongside the main results. IMDb's cast lists are incomplete for early films and ordered by billing, so truncating to the top three billed credits systematically drops unbilled supporting players — the truncation threshold is therefore treated as an experimental variable, with results reported at top-3 and top-8, and the effect on community structure reported as a robustness check. Finally, community labels are algorithmic constructs; the comparison with documented studio affiliations is qualitative and based on a 50-community manual sample, not on an exhaustive ground truth.

6. Conclusion (Planned)

The report will argue three things. Substantively, that Hong Kong cinema's collaboration network has a measurably different structure from the diffuse global network — higher clustering, stronger community separation, and communities that track production companies rather than genres. Methodologically, that the three workloads in this pipeline (scan, iterative graph, irregular mining) respond so differently to additional workers that "parallelise it" is not a single decision but three, each justified by a different property of the memory hierarchy. Practically, that a laptop-scale implementation with process-level parallelism and cache-aware blocking captures most of the available gain for this class of problem, without a cluster.

Future work will be scoped to what is genuinely reachable: extending the temporal analysis to a dynamic community detection method rather than fixed-label flux, incorporating box-office or budget data to test whether central nodes predict commercial outcomes, and replacing the heuristic region filter with a probabilistic record-linkage approach against the Hong Kong Film Archive catalogue.

7. Project Plan

7.1 Milestones

Week	Milestone	Owner	Deliverable
W1	Download and profile all five datasets; fix working-set definition; record row counts	A	Data profile memo, S0 complete
W2	S1 filter and join; chunk-size sweep; E1 results	A	E1 figures, Parquet shards

Week	Milestone	Owner	Deliverable
W3	S2 graph build; edge-truncation sensitivity; E2 results	A, B	Graph artefacts, E2 figures
W4	PageRank and label propagation; block-size sweep; E3 results	B	S3 complete, E3 figures
W5	Association rules; career clustering; community validation sample	C	S4 complete, Figs 7–9, 14
W6	Report drafting; figure standardisation; slide deck	D, all	Full draft to supervisor
W7	Presentation rehearsal; Q&A drill; final submission	All	Slides, report, datasets, cover sheet

7.2 Division of labour (to be confirmed by the group)

Member	Contribution	Share
A	Data acquisition, ingestion pipeline, E1–E2 experiments, scaling infrastructure	27%
B	Graph construction, PageRank and community detection, E3 experiment	27%
C	Association rule mining, clustering, validation sample, figures 3–9 and 14	26%
D	Report writing, figure standardisation, slides, Q&A preparation	20%

Percentages are indicative and must be finalised against actual effort before submission, since the marking scheme adjusts individual scores by stated contribution.

7.3 Risk register

Risk	Likelihood	Impact	Mitigation
Peak memory exceeds laptop capacity during join	Medium	High	Parquet sharding, column pruning, streaming reduction; fallback to a top-8 billing subset
PageRank speedup disappoints at high worker counts	Medium	Medium	Report the barrier-share analysis as a finding rather than a failure; this is itself the Amdahl result
Region filter yields too few titles to support graph analysis	Low	High	Relax to `HK` or `CN` or `TW`; add Hong Kong production companies; validate on 50 titles
Graph visualisation illegible at full scale	High	Low	Top-500 subgraph by PageRank, or decade-stratified sampling

Risk	Likelihood	Impact	Mitigation
Group member availability drops near deadline	Medium	Medium	Every stage has a documented owner and a written handover note in the repository

7.4 Repository layout

```
cs5488-hkfilm/
  data/            # raw downloads (gitignored), parquet shards
  src/ingest/      # S0-S1, chunk-parallel
  src/graph/       # S2-S3, blocked matvec, community detection
  src/rules/       # S4, candidate generation and counting
  src/bench/       # timing harness, scaling sweeps, memory sampling
  figures/         # generated, reproducible from scripts
  report/          # LaTeX or Word source
```

Benchmarks live in `src/bench` as first-class code rather than notebook cells, so that every speedup figure in the report can be regenerated by a single command. This matters for the Q&A: if an examiner asks how a number was produced, the answer should be a script path, not a recollection.

7.5 Mapping to the marking scheme

Criterion	Marks	How this project earns them
Real-world impact / creativity	5	A specific, under-studied industry analysed with a documented-phenomenon test, not a generic global dataset
Solid works and output amount	20	180 M rows across five sources, five pipeline stages, four experiments, fourteen figures
Technical depth and correctness	20	Graph centrality with sampling error bounds, stochastic community detection with stability testing, rules validated by chi-square
Parallel computing elements	20	Three distinct parallel strategies, strong and weak scaling, cache- and chunk-size sweeps, barrier and imbalance analysis, memory cost of parallelism
Use of written English	5	Structured, measured prose; consistent terminology; every figure referenced and interpreted
Presentation	30	One core claim per speaker, scaling curves as the visual anchor, rehearsed Q&A on methodology

Figures cited as approximate reflect the IMDb release at the time of download and must be re-measured and recorded in the final report for reproducibility.